

Lock Mechanism

- Multi-process solution for the Critical ~~Solution~~ ^{Section} problem
- Executes in User Mode
- Does NOT guarantee mutual exclusion

Typical Structure of a process:

```
while (1) {  
    ENTRY  
    Critical Section  
    EXIT  
    Remainder Section  
}
```

~~For a particular process~~

Every process will follow this format

```
do {  
    Acquire lock ENTRY  
    CS  
    Release lock EXIT  
}
```

P_i

CS

lock $\rightarrow$ 0/1 ✓

$P_0, P_1, P_2, \dots, P_n$

Lock Mechanism

```

while (1) {
Entry  [ ① while (lock == 1); // Check if CS is empty. Else stay in spinlock
        ② lock = 1; // Lock before entering CS

/* Critical Section */ (CS)
Exit [ ③ lock = 0 // Unlock after exiting CS

/* Remainder Section */ (RS)
}

```

Case 1

<u>P₀</u>	<u>P₁</u>	<u>lock</u>	<u>CS</u>	
1		0		Time
2		1		
CS		1	P ₀	
CS, *	1		P ₀	
3	1, *	0		
RS, *	2	1		
RS	CS, *	1	P ₁	
1	CS, *	1	P ₁	
1, *	CS	1	P ₁	

← Preemption)

Case 2

<u>P₀</u>	<u>P₁</u>	<u>Lock</u>	<u>CS</u>	
1, *		0		while (1) {
		0		① while (lock == 1);
1, *	1	0		② lock = 1;
		1		/* CS */
1, *	2	1		③ lock = 0
1, *	CS	1		/* RS */
2	CS, *	1		}
			P ₁ , P ₀	
<u>CS</u>	<u>CS, *</u>			

This happens because of Preemption between ① & ②

